Computational Economics

Lab: Working with AI Agents

Yaolang Zhong yaolang.zhong@e.u-tokyo.ac.jp
Graduate School of Economics, The University of Tokyo

Lab Overview

- **How an agent works**
  - The agent loop, the model, and the harness
  - Tools, CLI, and MCP
- **How to direct its work**
  - Prompts, context, and persistent project instructions
  - Reusable procedures with skills
- **How to check the result**
  - Files, execution records, and reproducible outputs
  - One task throughout: reproduce the maze figure

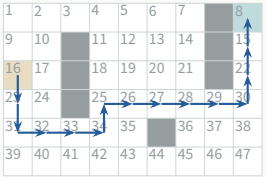
An Agent Working on the Maze Project

An **AI agent** uses a model to choose actions and tools, then uses the results to continue the task.

Our task

"Reproduce the maze solution and save a figure with clear start and goal labels."

- Read the model and plotting code.
- Run the solver; inspect errors or outputs.
- Revise the plotting code, then check the saved figure.



Yellow: start (16) Teal: goal (8)

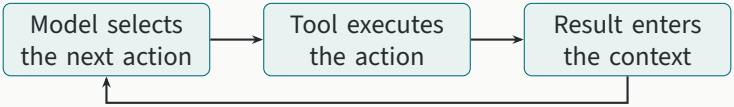


Figure 1: The agent loop repeats until completion or intervention.

The AI agent operates the project; VFI computes the maze policy.

The Model and the Harness

The **large language model (LLM)** generates responses and tool requests from its input. The **harness** is the software that runs the agent around that model.

- It assembles context, executes tool calls, and returns their results.
- It manages the session and enforces configured permissions.
- A **sandbox** restricts execution; an **approval** lets a person authorize a particular action.

In the maze task

- The model requests: read `maze.py`, then run Python.
- The harness checks access, runs the command in a working directory, and supplies the output.

Codex and Claude Code are agent applications. Desktop apps, IDE extensions, and CLI provide ways to interact with agents. Capabilities depend on the model and harness.

Reading: OpenAI, Codex as a platform.

A Prompt Specifies the Task

A **prompt** supplies instructions for the current task. Specify the result, relevant inputs, constraints, and checks.

A prompt for the maze project

> *Read* `README.md`*,* `maze.py`*,* `vfi.py`*, and* `figures.py`*. Reproduce the maze solution with* $\beta = 0.9$*.*
>
> *Add a script* `run_maze.py` *that saves* `results/maze.png`*. Label start cell 16 and goal cell 8 clearly. Preserve the model and solver.*
>
> *Run the script in the project environment. Check that the path reaches the goal and inspect the image. Report the changed files, command used, and checks performed.*

The requested output and checks make completion observable. If the environment is unavailable, ask the agent to report the blocker.

Reading: OpenAI, Prompt engineering.

Context Is What the Model Can Use Now

Context is the input available to a model for its next response.

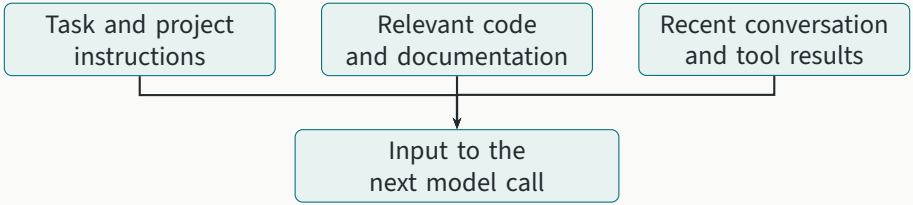


Figure 2: Context assembled for the next model call.

- Files become useful context when their relevant contents are loaded or retrieved.
- A **token** is a unit used to encode input and output. The **context window** limits how much a model call can accommodate.

Maze detail: reading `maze.py` reveals that cell 16 is Python index 15.

Keeping Context Useful

Context engineering means selecting and maintaining the information an agent needs as the task progresses.

- **Retrieve relevant material.** Read the maze code when needed; retrieve errors after a failed run.
- **Keep durable decisions.** Save parameters, indexing conventions, and the run command in project files.
- **Summarize long sessions.** **Compaction** condenses earlier context; summaries can omit details, so retain important facts in files.

Handoff note

"Maze reproduced at $\beta = 0.9$. Cell labels are one-based. Next: inspect the saved image."

Memory stores information for retrieval across sessions; its scope depends on the application.

Reading: Anthropic, Effective context engineering for AI agents.

Tools, the Terminal, and CLI

A **tool** is a callable operation with inputs and a returned result: reading a file, editing code, or executing a command.

- A **CLI** (command-line interface) accepts text commands.
- The **terminal** hosts the session; the **shell** interprets commands.

Commands in the maze working folder

```
python run_maze.py
git diff
```

Run the script after creating it; use `git diff` to review edits in a Git repository.

The working directory and active Python environment determine which files and packages are used.

An agent CLI accepts your tasks in a terminal. Agents in an IDE or app can also use shell tools.

Reading: VS Code, Terminal Basics.

MCP Connects an Agent to Capabilities

Model Context Protocol (MCP) is a standard for communication between an AI application and servers that expose tools and context.

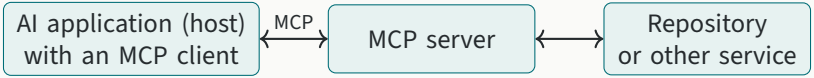


Figure 3: Connecting an application to a service through MCP.

- **Tools** perform operations; **resources** expose data; **prompts** provide reusable message templates.
- Servers run locally or remotely, with explicitly configured access.

Maze task: a repository MCP server could retrieve an issue describing the requested labels.

MCP standardizes the connection; the harness coordinates its use.

Reading: Model Context Protocol, Architecture overview.

Skills Package Reusable Procedures

A **skill** packages task instructions and optional scripts, references, or templates. In the Agent Skills format, its entry point is `SKILL.md`.

Example skill: reproduce a figure

```
reproduce-figure/
  SKILL.md
  scripts/
  references/
```

Procedure in the skill:

1. Identify the model and parameters.
2. Run the existing computation.
3. Save and inspect the figure.
4. Report the command and checks.

Reading: Agent Skills, Overview and Specification.

When the maze task matches:

- The agent discovers the skill through its name and description.
- It loads the full instructions when relevant.
- It uses available tools to carry out the procedure.

The same procedure can guide other figures. Skills provide instructions; tools and permissions determine which actions can run.

Project Instructions Keep Conventions Consistent

Project instructions store guidance for work in a particular project. For example, Codex reads `AGENTS.md`; supported filenames and scope vary by tool.

Possible project guidance for the maze

```
Use the existing project Python environment.
Keep model definitions separate from solver code.
Use one-based cell labels in figures.
Save generated images under results/.
Run the affected example after editing.
```

- The **task prompt** asks for this figure now.
- **Project instructions** describe conventions across tasks.
- A **skill** supplies a reusable procedure when the task calls for it.

Keep instructions short, current, and specific enough to check.

Reading: OpenAI, Custom instructions with AGENTS.md.

Checking an Agent's Work

An agent's completion message is a report. Check the underlying evidence.

1. **Inspect the edits.** Read the diff and check that edits match the plotting task.
2. **Reproduce the run.** Record the environment and command; confirm the output file exists.
3. **Check the result.** Verify that the path reaches cell 8; open the saved image and inspect the labels.
4. **Keep a recoverable version.** Commit reviewed code with Git to compare or reverse later edits.

A concrete check

The maze's `path` method detects cycles. Opening the saved PNG checks that labels are visible and correct.

Treat instructions inside retrieved files as content to assess. **Prompt injection** uses such content to redirect an agent away from its task.

Practice: Reproduce the Maze Figure

1. Open a copy of the Lab 3 working folder. Use an agent with file and terminal access, or attach the relevant files to a chat and run the returned code yourself.
2. Use the maze prompt. Let the agent inspect the files before editing, then review its tool actions and outputs.
3. Find one example of each: prompt, retrieved context, tool call, and tool result. Identify which application supplies the harness.
4. Run the new script and inspect `results/maze.png`. Review the changes before accepting them.

Result: a reproducible script, a labelled maze figure, and a short record of the command and checks.

Follow-up: save a recurring convention as a project instruction; turn a repeated procedure into a skill when your agent supports it.

The local maze task can be completed with file and shell tools. Add an MCP integration when the task needs an external service.

References and Further Reading

1. **OpenAI, Agent and Codex documentation. [Documentation]**
   - Codex as a platform: build on the open agent harness
   - Prompt engineering
   - Custom instructions with AGENTS.md
2. **Anthropic (2025), Effective context engineering for AI agents. [Technical article]**
   - Context engineering vs. prompt engineering

References and Further Reading

3. **Microsoft, Visual Studio Code: agent and terminal documentation. [Documentation]**
   - Understand agent harnesses
   - Terminal Basics
   - Secure AI-assisted development in VS Code
4. **Model Context Protocol contributors, Model Context Protocol documentation. [Documentation]**
   - Architecture overview
5. **Agent Skills contributors, Agent Skills documentation. [Documentation]**
   - Agent Skills Overview
   - Specification